

Introduction à l'apprentissage profond

TP4

S. Boudra & I. Yahiaoui

Dégradation du taux d'apprentissage

Lors du calcul du gradient stochastique, la mise à jour des poids et la minimisation de la fonction d'erreur dépendent de la valeur du taux d'apprentissage (souvent noté par α (alpha) ou bien η (eta)). Avec des valeurs très faibles de η , le réseau peut ne pas apprendre, avec des valeurs très élevées, le réseau peut ne pas converger vers une région du minimum de la fonction d'erreur.

Cela dit, il est important d'optimiser ce paramètre. Le principe consiste à choisir une valeur « optimale » pour l'apprentissage du réseau. Or ceci n'est pas pratique, et la valeur du taux d'apprentissage η est empirique (basée sur des expérimentations). Dans ce cas, on commence généralement l'apprentissage par une valeur initiale du taux d'apprentissage η , et on procède à sa dégradation au fur et à mesure qu'on avance dans l'entraînement.

Il existe plusieurs méthodes pour la dégradation du taux d'apprentissage¹. Dans ce qui suit, on va voir le `InverseTimeDecay()` et `ExponentialDecay()`. On peut aussi définir sa propre fonction de dégradation du taux d'apprentissage en la faisant passer à `LearningRateScheduler()`

Dégradation du taux d'apprentissage avec `InverseTimeDecay()`²

La dégradation par `InverseTimeDecay()` est accessible depuis le module `keras.optimizers.schedules.InverseTimeDecay` est donné par la formule suivante, où 'initial_learning_rate' est la valeur initiale du taux d'apprentissage qu'on souhaite dégrader durant l'apprentissage ; 'decay_rate' est le taux avec lequel on baisse la valeur du taux d'apprentissage ; et 'decay_step' est le nombre de pas à partir duquel on dégrade le taux d'apprentissage

```
def decayed_learning_rate(step):  
    return initial_learning_rate / (1 + decay_rate * step / decay_step)
```

¹ https://keras.io/api/optimizers/learning_rate_schedules/

² https://keras.io/api/optimizers/learning_rate_schedules/inverse_time_decay/

Avec keras ça donne la partie du code suivante :

```
from keras.optimizers.schedules import InverseTimeDecay

# Inverse Time Decay
lr_schedule = InverseTimeDecay(0.003, 100, 0.5)
grad_descent = SGD(learning_rate=lr_schedule)
# compile the model
model.compile(optimizer=grad_descent, loss="categorical_crossentropy",
metrics=["accuracy"])
# train the model
model.fit(x_train, y_train, validation_data=(x_test, y_test),
batch_size=batch_size, epochs=epochs, verbose=1)
```

Dégradation du taux d'apprentissage avec `ExponentialDecay()`³

La dégradation par `ExponentialDecay()` est accessible depuis le module `keras.optimizers.schedules.ExponentialDecay` est donné par la formule suivante, où 'initial_learning_rate' est la valeur initiale du taux d'apprentissage qu'on souhaite dégrader durant l'apprentissage ; 'end_learning_rate' valeur après laquelle on ne dégrade plus le taux d'apprentissage ; 'power' un réel utilisé pour calculer le decay_rate ; et 'decay_step' est le nombre de pas à partir duquel on dégrade le taux d'apprentissage

```
def decayed_learning_rate(step):
    step = min(step, decay_steps)
    return ((initial_learning_rate - end_learning_rate) *
            (1 - step / decay_steps) ^ (power)
            ) + end_learning_rate
```

Avec keras ça donne la partie du code suivante :

³ https://keras.io/api/optimizers/learning_rate_schedules/exponential_decay/

```

from keras.optimizers.schedules import ExponentialDecay

init_lr = 0.001
lr_schedule = ExponentialDecay(init_lr, decay_steps=500, decay_rate=0.98,
staircase=True)
optimizer = SGD(learning_rate=lr_schedule)
# compile the model
model.compile(optimizer=optimizer, loss="categorical_crossentropy",
metrics=["accuracy"])
# train the model
model.fit(x_train, y_train, validation_data=(x_test, y_test),
batch_size=batch_size, epochs=epochs, verbose=1)

```

Dégradation du taux d'apprentissage avec `LearningRateScheduler()`⁴

L'application d'une méthode programmée de dégradation du taux d'apprentissage lors de l'entraînement du réseau est effectuée par l'attribut `callbacks` qui prend comme valeur une liste d'objet de la classe `LearningRateScheduler`⁵.

Avec keras ça donne la partie du code suivante, où la fonction `custom_lr_scheduler()` définit une fonction de dégradation du taux d'apprentissage :

```

def custom_lr_scheduler(epoch, current_lr):
    return current_lr / 3

lr_schedule = LearningRateScheduler(custom_lr_scheduler)
grad_descent = SGD(learning_rate=0.001)
# compile the model
model.compile(optimizer=grad_descent, loss="categorical_crossentropy",
metrics=["accuracy"])
# train the model
model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
validation_data=(x_test,y_test), callbacks=[lr_schedule], verbose=1)

```

Qst: Compléter les différents exemples de dégradation sur les bases traitées dans les TPs précédents.

⁴ https://keras.io/api/callbacks/learning_rate_scheduler/